

TouchLess AI — Adaptive Hand Gesture Utility Control

Project Report

TouchLess AI — Adaptive Hand Gesture Utility Control

Project Report

1. Introduction

TouchLess AI is a real-time adaptive hand gesture utility control system developed using Computer Vision and Human-Computer Interaction concepts. The primary objective of the project is to enable users to interact with system utilities through touchless hand gestures captured using a webcam. The system utilizes OpenCV and MediaPipe-based hand landmark detection to recognize specific finger gestures and translate them into system-level actions such as volume adjustment, brightness control, screenshot capture, and bi-directional scrolling.

The project focuses on providing a lightweight, low-latency, and user-friendly interaction framework without requiring specialized hardware. Unlike traditional input devices, the proposed system enables contactless interaction through real-time hand tracking and gesture analysis. Personalized calibration and threshold tuning techniques were additionally incorporated to improve gesture recognition stability across different hand sizes and webcam distances.

2. Objectives

The major objective of the project is to develop a touchless utility control framework capable of performing real-time system interactions through hand gestures. The system aims to achieve accurate hand landmark extraction using MediaPipe, reliable gesture recognition using landmark distance mapping, and seamless execution of utility operations such as volume control, brightness regulation, screenshot capture, and scrolling.

Another objective of the project is to improve interaction robustness by integrating personalized gesture calibration and false-trigger prevention mechanisms. The project also focuses on maintaining modularity and low computational overhead to support smooth real-time execution on standard laptop hardware.

3. Problem Statement

Traditional system interaction methods rely heavily on physical input devices such as keyboards, touchpads, and mice. These interaction methods may not always be convenient in scenarios requiring contactless operation or accessibility-focused interaction. Existing gesture-based systems often suffer from unstable gesture recognition, false activations, and inconsistent performance across varying hand sizes and camera distances.

The proposed TouchLess AI system addresses these limitations by implementing adaptive gesture calibration, strict gesture separation logic, and landmark-based motion analysis for stable real-time interaction. The system provides an intuitive and touchless method for controlling common system utilities using only a webcam.

4. Technologies Used

The project was implemented using Python as the primary programming language due to its extensive support for Computer Vision and automation libraries. OpenCV was used for webcam access, frame processing, and real-time visualization. MediaPipe Hands was integrated for extracting 21 hand landmarks from webcam frames with high-speed real-time tracking capabilities.

PyAutoGUI was used for system automation tasks such as scrolling and screenshot capture, while the screen-brightness-control library was utilized for adjusting display brightness dynamically. NumPy was additionally used for numerical operations and distance calculations during gesture recognition.

5. System Architecture

The system architecture follows a modular real-time Computer Vision pipeline. Initially, webcam frames are captured continuously through OpenCV and passed to the MediaPipe Hands framework for landmark extraction. The extracted landmark coordinates are then processed using gesture recognition logic based on finger-state analysis and landmark distance mapping.

After identifying the corresponding gesture, the detected action is forwarded to the system automation layer, where utility operations such as brightness control, volume adjustment, screenshot capture, or scrolling are executed. A personalized calibration module is additionally integrated to dynamically adjust gesture thresholds according to individual user hand ranges.

Webcam Input



Hand Detection using MediaPipe



Landmark Extraction



Gesture Recognition



Gesture Mapping



System Utility Execution

6. Modules Description

The Hand Tracker module is responsible for detecting and tracking hand landmarks in real time using the MediaPipe Hands framework. The module extracts 21 key landmark coordinates from webcam frames and provides positional data required for gesture analysis.

The Gesture Logic module processes the extracted landmark coordinates and performs gesture recognition using finger-state analysis and Euclidean distance mapping techniques. The module identifies predefined gesture patterns and maps them to system utility operations.

The Mouse Control module handles scrolling operations through motion-based gesture tracking using PyAutoGUI. Vertical hand movement is analyzed to determine upward and downward scrolling directions.

The System Control module performs system automation tasks such as volume adjustment, brightness regulation, and screenshot capture. This module acts as the bridge between gesture recognition and operating system-level utility execution.

The Calibration module dynamically learns user-specific gesture ranges during application startup. Personalized minimum and maximum distance thresholds are captured to improve robustness across varying hand sizes, camera distances, and lighting conditions.

The modular architecture improves maintainability, scalability, and debugging efficiency while enabling stable real-time gesture interaction.

7. Functionalities Implemented

The project implements multiple real-time touchless utility control functionalities using predefined hand gestures.

The volume control functionality uses thumb and index finger distance mapping to dynamically adjust system volume. Increasing the distance between the fingers raises the volume level, while decreasing the distance lowers the volume.

Brightness control is implemented using thumb and pinky finger distance analysis. The system continuously maps the distance between the two landmarks to screen brightness values for adaptive brightness regulation.

Screenshot capture functionality is triggered using a thumbs-up gesture. Upon recognition, the system automatically captures and stores screenshots with timestamped filenames.

Bi-directional scrolling is implemented using index and middle finger gestures combined with vertical hand movement tracking. Upward hand motion scrolls upward, while downward motion scrolls downward.

Additional false-trigger prevention logic and strict gesture separation mechanisms were integrated to improve interaction stability and reduce unintended activations.

8. Methodology

The implementation process began with real-time webcam integration using OpenCV. Each captured frame was processed using the MediaPipe Hands framework to extract 21 hand landmarks representing finger joint coordinates.

The extracted landmarks were analyzed using finger-state logic and Euclidean distance calculations to determine gesture patterns. Dynamic distance mapping techniques were implemented to convert finger spacing into utility control percentages for brightness and volume operations.

To improve interaction robustness, a personalized calibration system was developed. During startup, the system records user-specific minimum and maximum gesture distances and utilizes them for adaptive threshold mapping.

Motion-based gesture analysis was additionally incorporated for implementing bi-directional scrolling functionality. Finally, PyAutoGUI and system-level automation libraries were integrated to execute utility operations in real time.

9. Advantages

The proposed system provides a touchless and intuitive method of interacting with system utilities using only a webcam. The lightweight architecture enables low-latency real-time performance without requiring specialized hardware.

Personalized calibration improves robustness across different users, while strict gesture separation reduces false activations. The modular implementation additionally supports scalability and future feature integration.

10. Limitations

The system performance may vary under poor lighting conditions or unstable webcam positioning. Gesture recognition accuracy can also be affected by extreme hand rotations and partial landmark occlusions.

The current implementation supports only single-hand interaction and predefined gesture mappings. The project additionally depends on webcam frame quality for stable landmark extraction.

11. Future Enhancements

Future enhancements may include AI-based gesture classification using deep learning models, multi-hand gesture support, custom gesture configuration, and voice feedback integration.

The system can additionally be extended into accessibility-oriented communication systems such as sign-language recognition frameworks and gesture-to-speech applications.

Deployment as a cross-platform desktop or web-based application using Streamlit or Electron can further improve usability and accessibility.

12. Conclusion

TouchLess AI successfully demonstrates a real-time adaptive hand gesture utility control framework using Computer Vision and Human-Computer Interaction techniques. The project integrates OpenCV and MediaPipe-based hand landmark extraction with gesture recognition logic for performing real-time utility operations such as volume control, brightness adjustment, screenshot capture, and bi-directional scrolling.

The incorporation of personalized calibration, threshold tuning, motion-based gesture analysis, and false-trigger prevention mechanisms significantly improved interaction stability and usability. The project

showcases the practical application of landmark-based gesture analysis for touchless human-computer interaction while maintaining low computational overhead and real-time responsiveness.

The developed system provides a strong foundation for future AI-driven gesture recognition and accessibility-oriented interaction systems.

README.md

TouchLess AI — Adaptive Hand Gesture Utility Control

A real-time adaptive hand gesture utility control system developed using OpenCV and MediaPipe for controlling system utilities such as volume, brightness, screenshot capture, and bi-directional scrolling through webcam-based gesture recognition.

Features

- Real-time hand landmark detection
 - Adaptive volume control
 - Dynamic brightness adjustment
 - Screenshot capture
 - Motion-based bi-directional scrolling
 - Personalized calibration system
 - False-trigger prevention
 - Real-time gesture tracking
-

Technologies Used

- Python
 - OpenCV
 - MediaPipe
 - PyAutoGUI
 - screen-brightness-control
 - NumPy
-

Installation

Clone Repository

git clone <repository-link>

```
cd TouchLess-AI
```

Create Virtual Environment

```
python -m venv venv
```

Activate Environment

```
venv\\Scripts\\activate
```

Install Dependencies

```
pip install -r requirements.txt
```

Run Project

```
python main.py
```

Gesture Controls

Gesture	Function
Thumb + Index	Volume Control
Thumb + Pinky	Brightness Control
Thumbs Up	Screenshot Capture
Index + Middle	Scroll Up / Down

Calibration Process

The application initially performs gesture calibration to dynamically capture user-specific minimum and maximum gesture ranges. This improves interaction stability across varying hand sizes and webcam distances.

Output

- Real-time webcam-based gesture tracking
 - Dynamic utility control
 - Screenshot capture and storage
 - Adaptive brightness and volume adjustment
 - Motion-based scrolling
-